Flood Prediction System Using Machine Learning

Project Description:

The **Flood Prediction System Using Machine Learning** is an intelligent web application that predicts the likelihood of flood occurrence based on weather and rainfall parameters. The system uses machine learning algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost to analyze historical flood-related data and generate accurate predictions. The best-performing model is deployed using Flask, enabling users to enter weather information and receive instant flood risk predictions through a simple and user-friendly interface. By providing early warnings, the system helps authorities and citizens take preventive measures, minimizing the impact of floods and improving disaster preparedness.

Problem Statement

Floods are one of the most destructive natural disasters, causing significant loss of life, property, and infrastructure. Traditional flood prediction methods often require expert analysis and may not provide timely warnings. There is a need for an automated, data-driven system that accurately predicts flood occurrence based on weather and rainfall parameters, enabling early preventive actions and improving disaster management.

Scenarios:

Scenario 1: Resident Flood Prediction

A resident living in a flood-prone region enters weather information such as temperature, humidity, cloud cover, and rainfall into the web application. The system analyzes the data using the trained machine learning model and instantly predicts whether there is a possibility of flooding.

Scenario 2: Disaster Management Planning

A disaster management officer monitors changing weather conditions. By entering the latest environmental parameters into the system, the model predicts flood risk, allowing authorities to prepare rescue teams, issue warnings, and organize evacuation plans before flooding occurs.

Scenario 3: Government Weather Monitoring

Government officials use the application to analyze weather conditions across different regions. The system provides rapid flood risk predictions, helping authorities allocate emergency resources efficiently and reduce damage caused by floods.

Objectives

- To collect and preprocess flood-related weather and rainfall data.
- To perform exploratory data analysis (EDA) for understanding the dataset.
- To train multiple machine learning algorithms including Decision Tree, Random Forest, KNN, and XGBoost.
- To compare the performance of different models using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

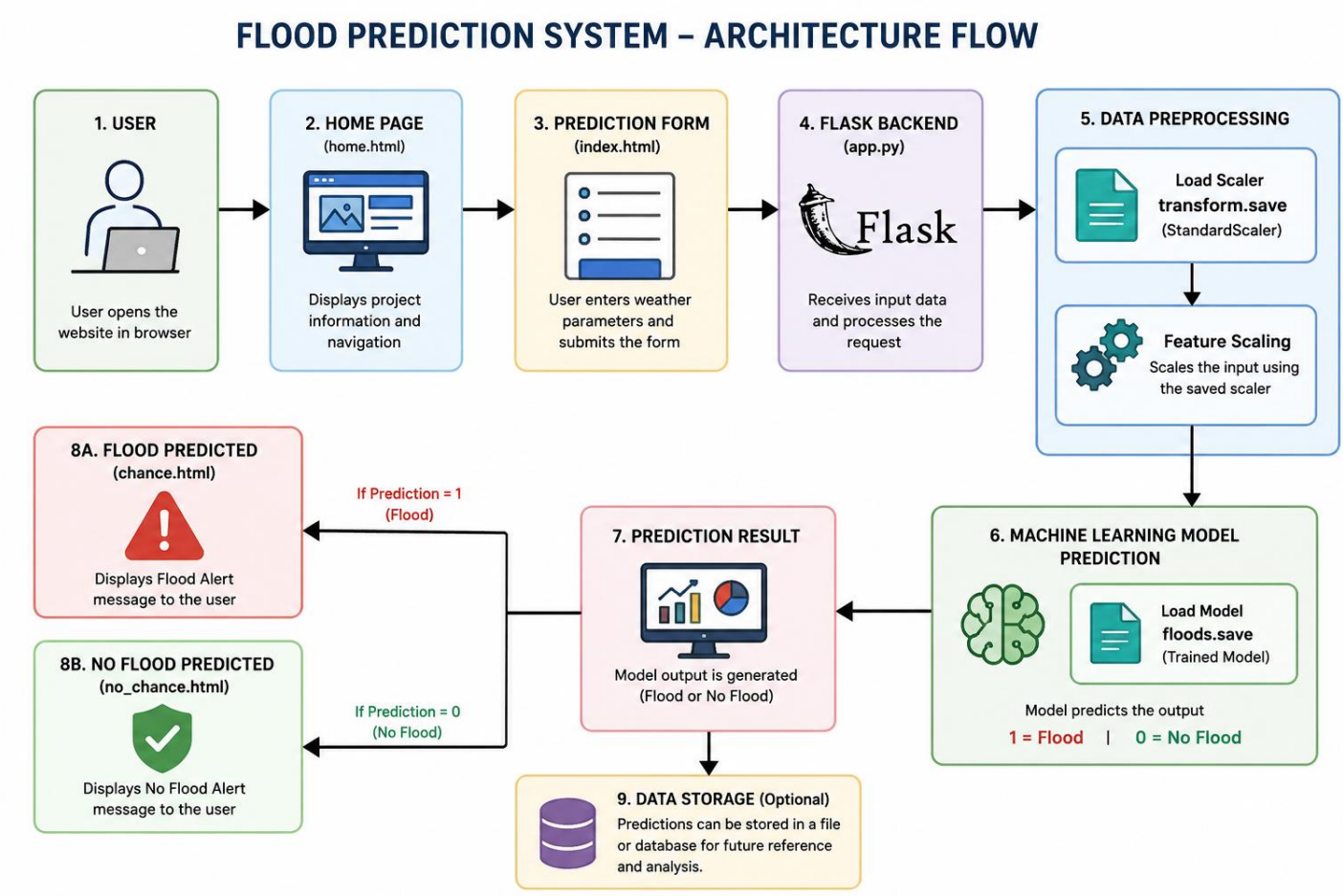
- To select the best-performing model for deployment.
- To develop a Flask-based web application for real-time flood prediction.
- To provide users with an easy-to-use interface for early flood risk assessment.

Scope

The scope of this project is limited to predicting flood occurrence (Flood / No Flood) using weather and rainfall parameters available in the dataset. The project demonstrates the complete machine learning workflow, including data preprocessing, model training, evaluation, and deployment through a web application. The system is intended for educational purposes and should not replace official flood forecasting systems without further validation using real-time environmental data.

Technical Architecture:

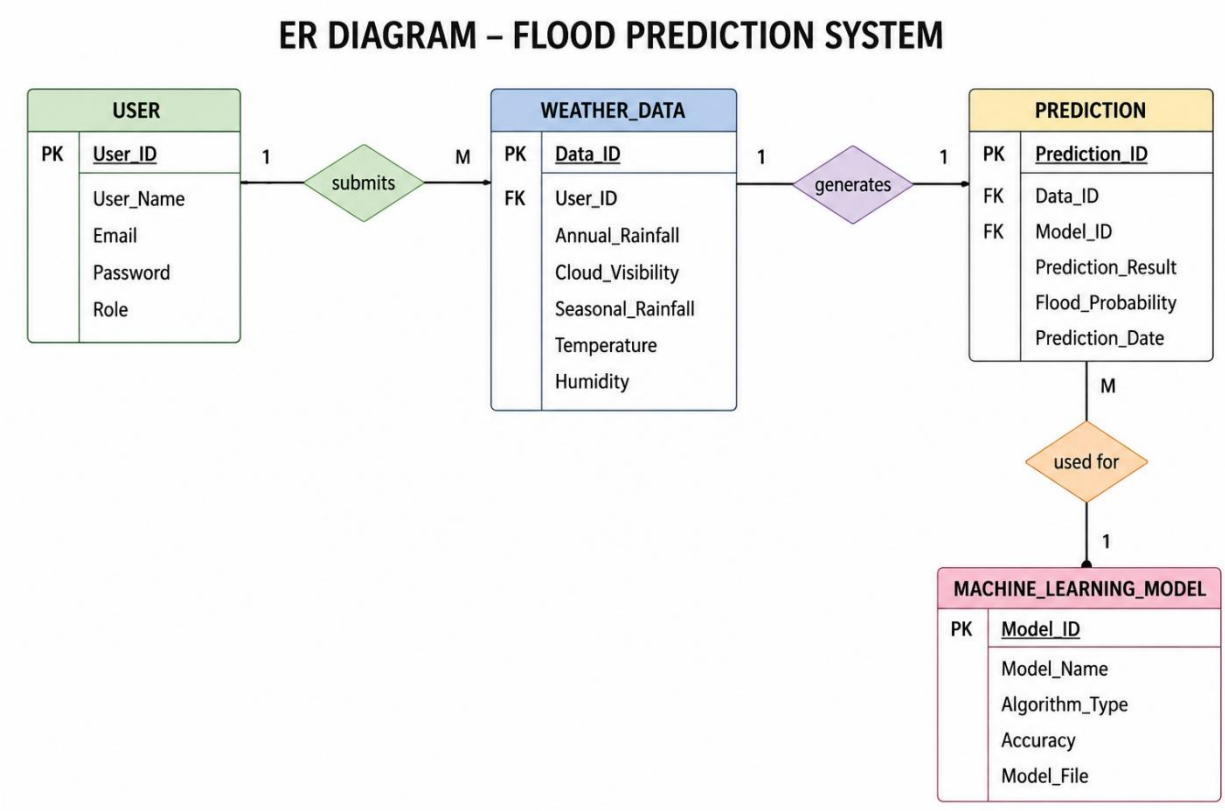
Architecture Flow



ER Diagram

Description:

The **Flood Prediction System** follows a modular machine learning architecture to provide accurate and real-time flood risk predictions based on weather and rainfall parameters. Users enter environmental information such as **Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan–Feb Rainfall, Mar–May Rainfall, Jun–Sep Rainfall, Oct–Dec Rainfall, Average June Rainfall, and Subdivision** through a **Flask-based web application**. The input data is validated and preprocessed using the same **StandardScaler** that was applied during model training. The processed data is then passed to the **trained Decision Tree machine learning model**, which predicts whether there is a possibility of flooding. Based on the prediction, the system displays either a **Flood Chance** or **No Flood Chance** result page to the user. This architecture enables quick and reliable flood prediction, helping authorities and citizens take timely preventive measures and improve disaster preparedness.



Pre-requisites

- Python Programming Proficiency (Python 3.x)
- Machine Learning Fundamentals — scikit-learn, XGBoost
- Data Handling Libraries — Pandas, NumPy
- Web Framework Knowledge — Flask, for deployment of the prediction interface
- Version Control with Git

Project Workflow:

Milestone 1: Data Collection and Understanding :

- **Activity 1.1:** Import the flood dataset (flood dataset.xlsx) and inspect its structure, columns, shape, and data types.
- **Activity 1.2:** Perform Exploratory Data Analysis (EDA) to understand the distribution of weather and rainfall features and analyze the class balance between Flood and No Flood cases.
- **Activity 1.3:** Identify numerical and categorical features relevant for flood prediction and study the correlation between features and the target variable.

Milestone 2: Data Preprocessing and Feature Engineering

- **Activity 2.1**: Verify the dataset for missing values and handle them using appropriate techniques such as median or mode imputation.
- **Activity 2.2**: Check for duplicate records and remove them to improve data quality.
- **Activity 2.3:** Apply Label Encoding to categorical variables (if present) to convert them into numerical values suitable for machine learning algorithms.
- **Activity 2.4:** Detect and treat outliers in numerical features using the Interquartile Range (IQR) capping method.
- **Activity 2.5:** Separate the dataset into feature variables (X) and target variable (Flood).
- **Activity 2.6:** Split the dataset into training (80%) and testing (20%) sets using train_test_split with stratified sampling.
- **Activity 2.7:** Apply StandardScaler to normalize numerical features before training scale-sensitive machine learning models.

Milestone 3: Model Building

- **Activity 3.1:** Train a Decision Tree Classifier to predict flood occurrence based on weather parameters.
- **Activity 3.2:** Train a Random Forest Classifier to improve prediction performance through ensemble learning.
- **Activity 3.3:** Train a K-Nearest Neighbors (KNN) Classifier for instance-based flood prediction.
- **Activity 3.4:** Train an XGBoost Classifier to leverage gradient boosting for enhanced predictive performance.

Milestone 4: Model Evaluation and Selection

- **Activity 4.1: Evaluate each trained model using Accuracy, Precision, Recall, and F1-Score.**
- **Activity 4.2: Generate a Confusion Matrix for each model to analyze prediction performance.**
- **Activity 4.3: Compare all trained models based on evaluation metrics and overall prediction capability.**
- Activity 4.4: Select the best-performing model for deployment. Based on the evaluation results (presented in Section 5 – Results), the Decision Tree Classifier achieved the highest accuracy of 95.65% and was selected as the final model for deployment.

Milestone 5: Deployment

- **Activity 5.1:** Save the trained Decision Tree model (floods.save) and the fitted StandardScaler (transform.save) using Joblib.
- **Activity 5.2:** Develop a Flask-based web application with an HTML, CSS, and JavaScript front-end that accepts weather and rainfall parameters from users.
- **Activity 5.3:** Integrate the trained machine learning model with the Flask backend (app.py) to provide real-time flood predictions through the /predict endpoint.
- **Activity 5.4:** Display prediction results on dedicated web pages indicating either **Flood Predicted** or **No Flood Predicted**.
- **Activity 5.5:** Test the complete application to verify correct operation from user input to prediction output.

Full implementation details, source code, and screenshots of the deployed web application are provided in Section 6 (Web Application).

Methodology

Dataset Description

Attribute	Details
Dataset Name	flood dataset.xlsx
Type	Structed
Target Variable	Flood (0 = No Flood, 1 = Flood)
Feature Types	Numerical Weather and Rainfall Parameters

Data Preprocessing Steps

- Data Loading and Inspection: The flood dataset (115 records, 11 columns) was loaded using Pandas. The dataset was examined using head(), info(), describe(), and shape() to understand its structure and statistical properties.
- Missing Value Verification: The dataset was checked for missing values using isnull().sum(). No missing values were found.
- Duplicate Checking: Duplicate records were identified using duplicated() and removed where necessary to improve data quality.
- Categorical Encoding: Any categorical attributes (if present) were converted into numerical form using Label Encoding.
- Outlier Treatment: Outliers in numerical features were identified using the Interquartile Range (IQR) method and capped to reduce their impact on model performance.
- Feature Selection: The target variable (Flood) was separated from the input weather parameters.
- Train-Test Split: The dataset was divided into training (80%) and testing (20%) subsets using stratified sampling to preserve class distribution.
- Feature Scaling: StandardScaler was applied to normalize numerical features before model training.

Algorithms Used

- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbors (KNN)
- XGBoost Classifier

Evaluation Metrics

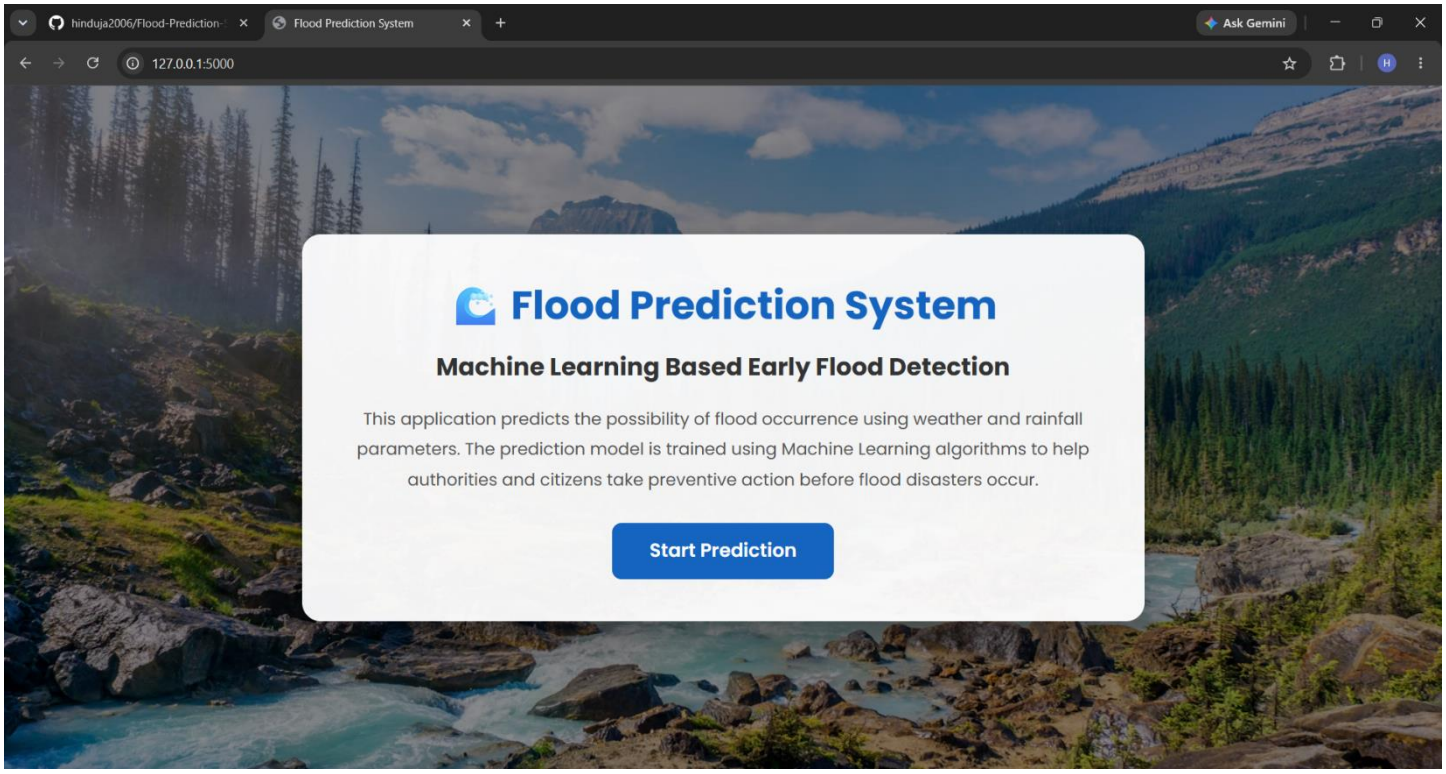
- Accuracy — Measures the overall percentage of correctly predicted flood and non-flood cases.
- Precision — Measures how many predicted flood cases are actually floods.
- Recall — Measures how many actual flood cases are correctly identified.
- F1-Score — Harmonic mean of Precision and Recall, providing a balanced evaluation metric.
- Confusion Matrix — Displays the number of correct and incorrect predictions for both flood and non-flood classes.
- Model Comparison — Compares the performance of Decision Tree, Random Forest, KNN, and XGBoost to identify the most accurate model.

Web Application

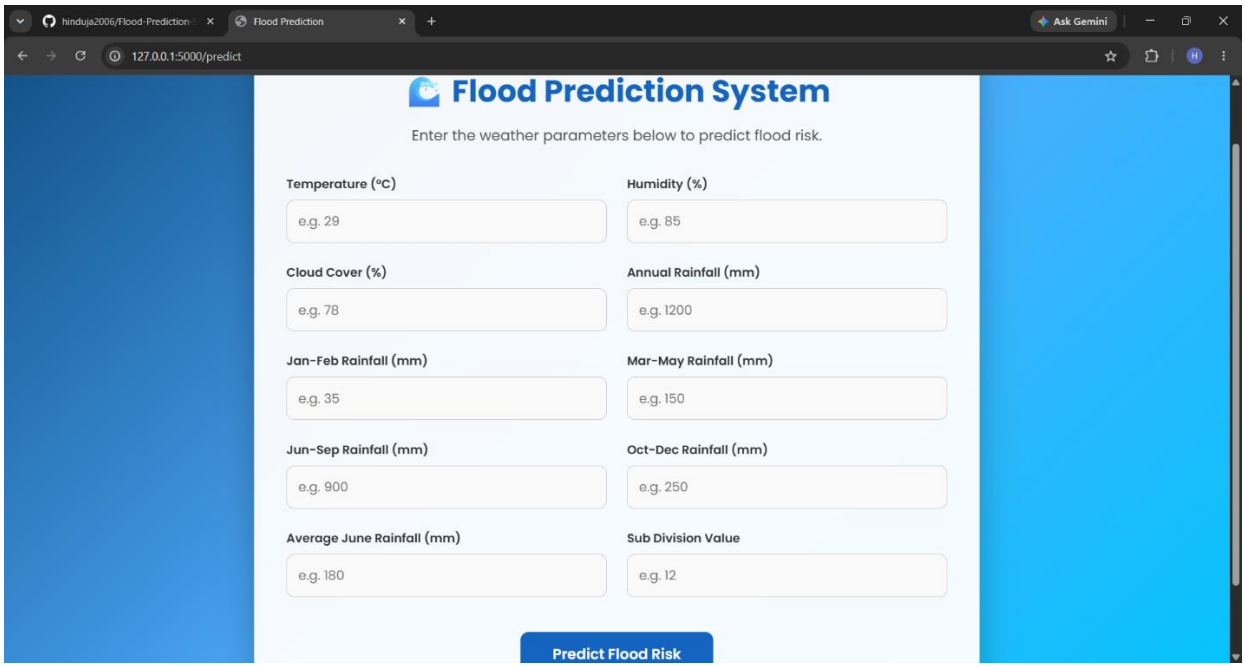
The trained Decision Tree model has been deployed as an interactive web application named "Flood Prediction System", developed using Flask as the backend and HTML, CSS, and JavaScript for the frontend. The application allows users to enter weather and rainfall parameters through an easy-to-use interface and instantly predicts whether there is a possibility of a flood (Flood / No Flood).

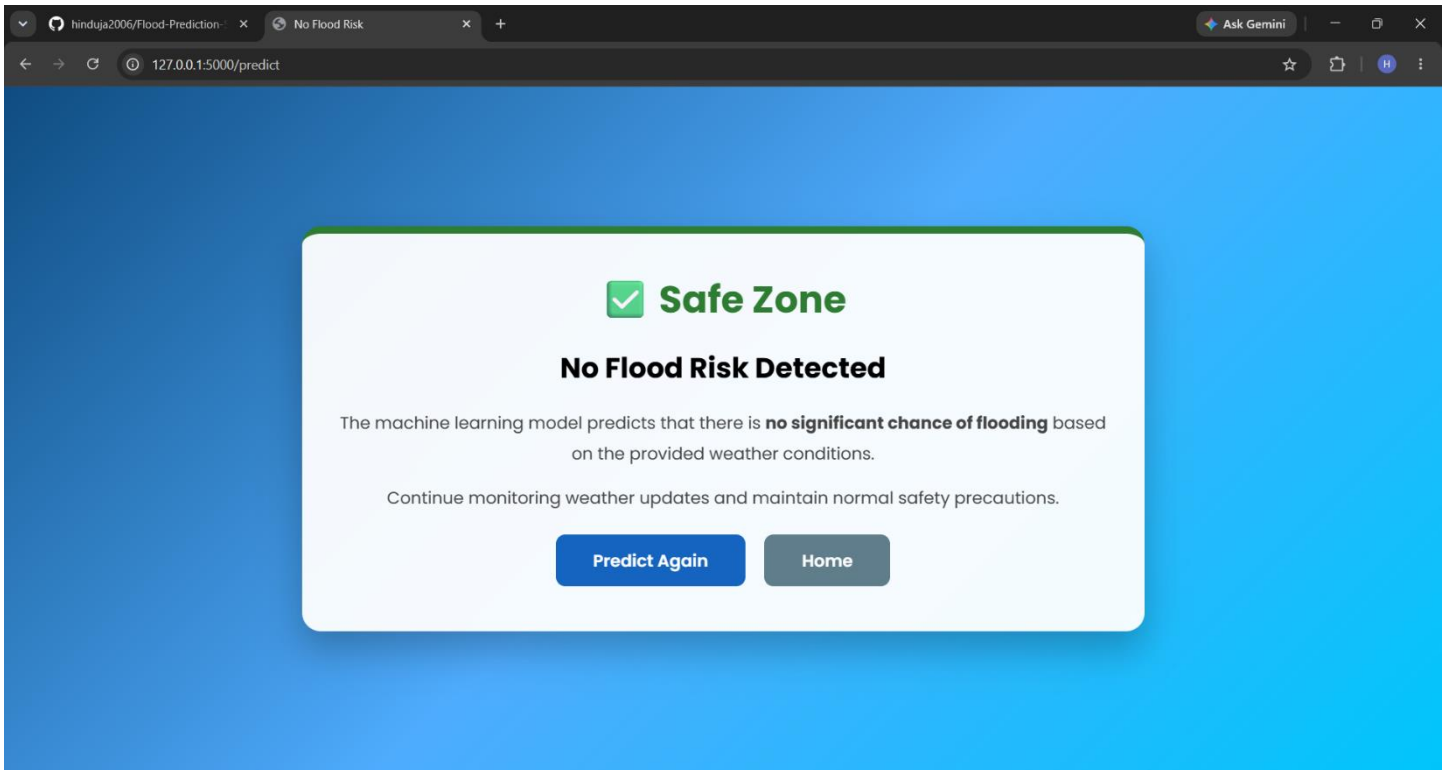
The application provides a simple and user-friendly interface, enabling quick prediction of flood risk to support early warning and disaster preparedness.

Application Form



Prediction of Result





Conclusion and Future Scope

Conclusion

The Flood Prediction System Using Machine Learning successfully demonstrates how machine learning can be used to predict flood occurrence based on weather and rainfall parameters. Multiple classification algorithms, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, were trained and evaluated. Among these, the Decision Tree Classifier achieved the highest accuracy of 95.65% and was selected for deployment.

The trained model was integrated into a Flask-based web application, allowing users to input weather parameters and receive real-time flood predictions through an intuitive interface. The system can assist disaster management authorities, government agencies, and citizens by providing early flood warnings, enabling timely preventive measures and reducing the impact of flood-related disasters.

Overall, the project demonstrates that machine learning is an effective approach for improving flood prediction accuracy and supporting disaster preparedness.

Future Scope

The proposed Flood Prediction System can be further enhanced with several advanced features to improve prediction accuracy and practical usability.Integrate larger and more diverse historical flood datasets collected from different geographical regions.Deploy the application on cloud platforms such as AWS, Microsoft Azure, or Google Cloud Platform for improved scalability and availability.Incorporate real-time weather data using APIs from meteorological services for live flood prediction.Apply advanced deep learning models such as LSTM, GRU, or Hybrid CNN-LSTM for time-series flood forecasting.Integrate satellite imagery and GIS (Geographical Information System) data to improve prediction accuracy.Implement Explainable AI (XAI) techniques such as SHAP or LIME to explain why a flood prediction was made.Develop a mobile application to provide instant flood alerts and notifications to users.Continuously retrain the model using newly collected weather and flood data to improve performance over time.Add user authentication and secure data storage for government and disaster management organizations.Extend the system to support prediction of other natural disasters such as landslides, cyclones, droughts, and heavy rainfall events.